

目录

第 1 章 ISE 14.7 简单教程 1

1.1 新建工程2

1.1.1 新建源代码2

1.1.2 添加源代码5

1.1.3 代码模板6

1.2 添加约束6

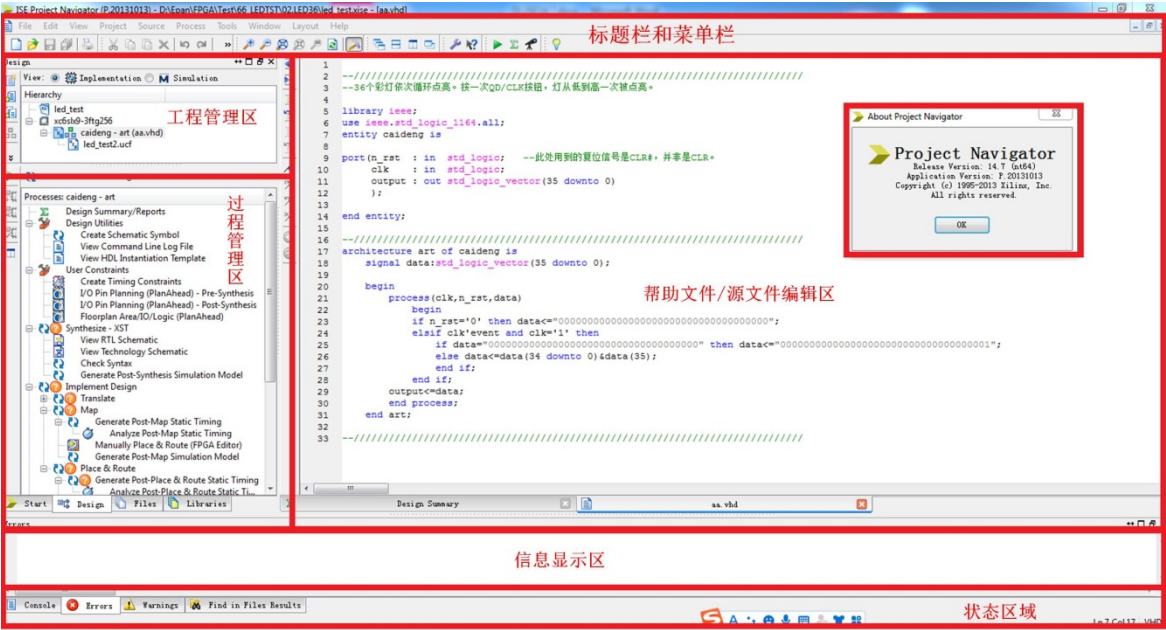
1.3 综合与实现9

1.4 程序下载10

第 1 章 ISE 14.7 简单教程

本章只简单介绍 XILINX FPGA 开发所需要的简单流程。ISE 14.7 的主要功能包括设计输入、综合、仿真和下载，含 FPGA 开发的全部过程，从功能上讲，可以不借助第三方的 EDA 软件，也可以和其他的仿真软件相互关联。

打开 ISE 14.7 软件，如图所示：ISE 14.7 的主窗口

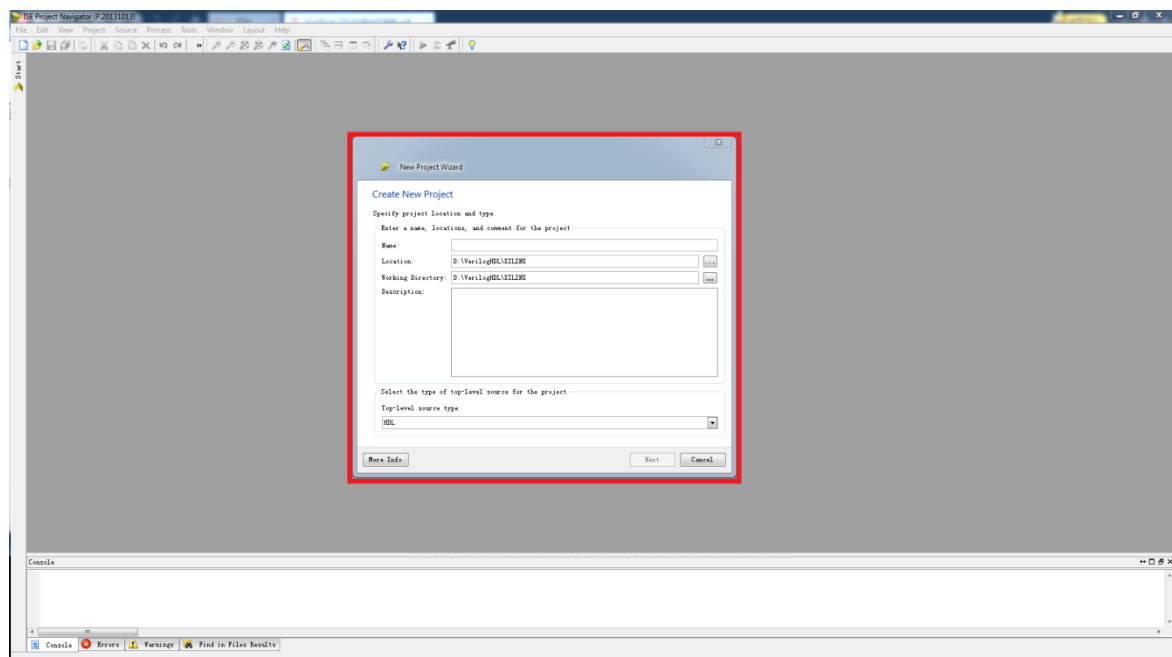


首先打开 ISE14.7 软件，每次启动时，ISE 14.7 都会默认到最近使用过的工程界面。

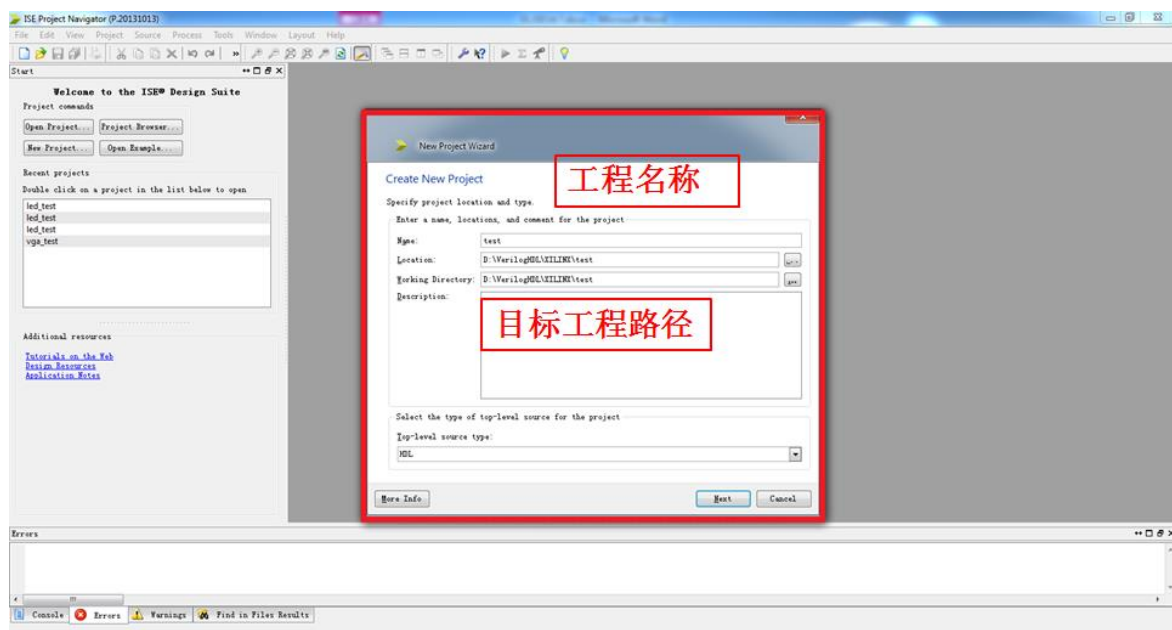
下面以一个彩灯循环点亮程序的设计为例，说明如何使用 ISE 14.7 创建工程并进行调试以及如果使用 XC6SLX9-3FTG256 进行配置。

1.1 新建工程

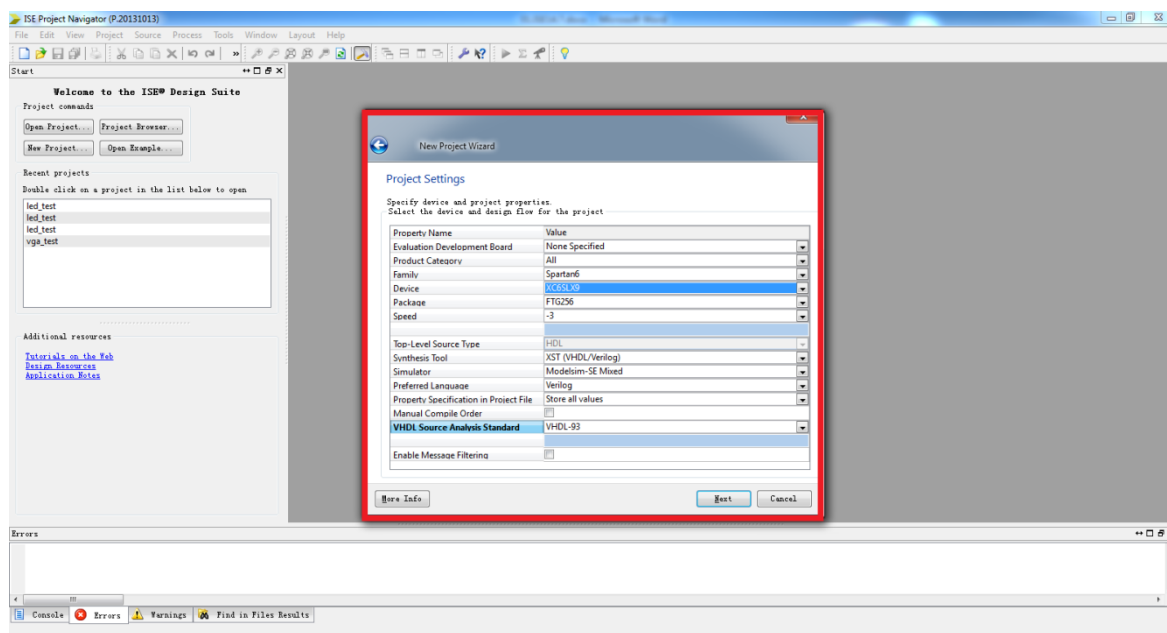
步骤一：在 ISE Project Navigator 开发环境里选择菜单 File->New Project。



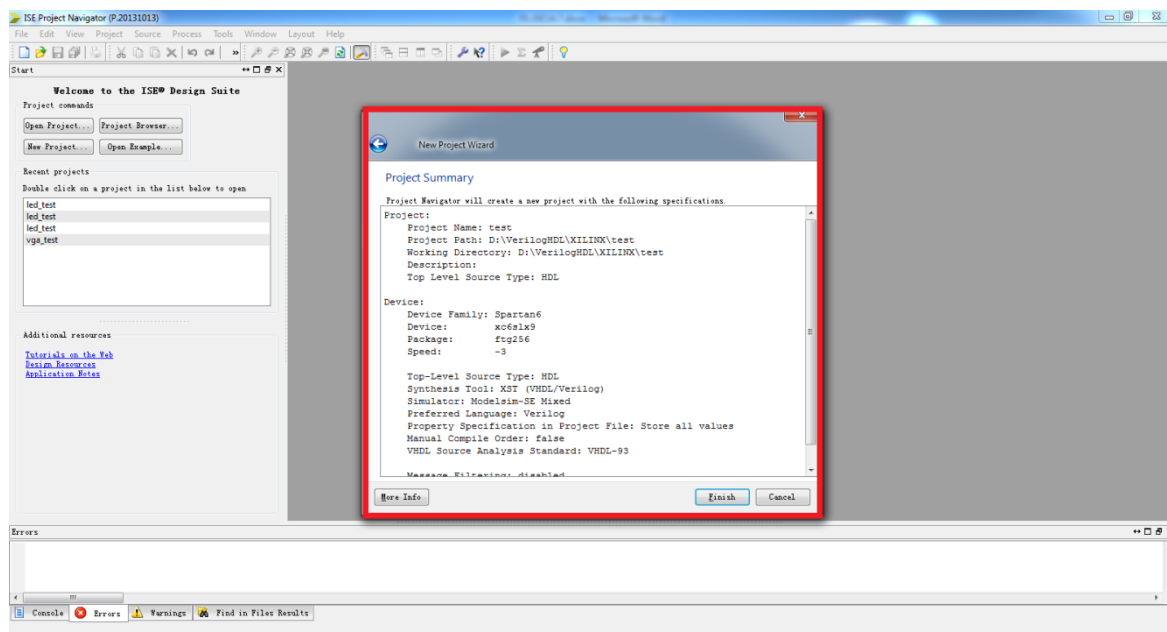
步骤二：在弹出的对话框中输入工程名称和目标工程路径，我们这里取一个 test 的工程名，目标工程路径可以自己选择，选择 Next 按钮。（注意：在所有工程文件的建立和保存过程中，保存路径和文件名称等都不能使用中文、空格或者其他特殊字符，以免造成编译不过或报错等现象）；



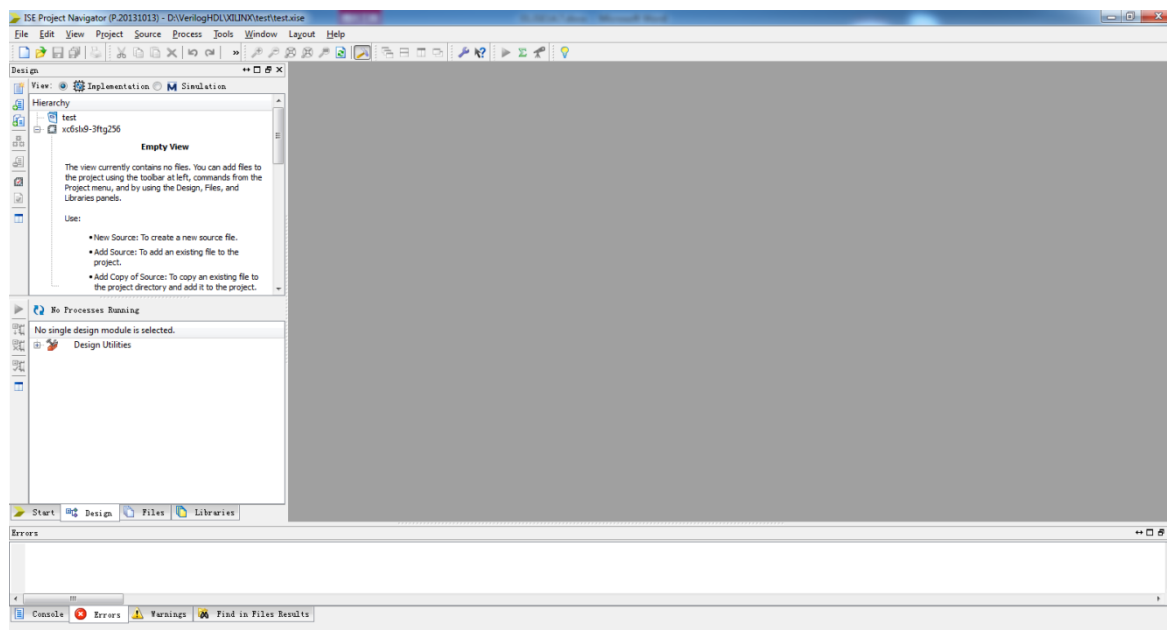
步骤三：在接下来的对话框中选择试验箱核心板所用的 FPGA 器件型号已经进行工程的一些配置。这里可以指定所选用的芯片，主要包括芯片系列、型号、封装以及速度等，这些内容在所选的芯片上都有标注，根据芯片选择即可，这里如下图所示的 Spartan6 系列的 XC6SLX9, 封装为 FTG256, 速度等级为-3，Preferred Language 对应语言类型，使用 Verilog/VHDL 语言编程均可以。



步骤四：单击 Next 按钮，出现工程汇总对话框，里面显示了所创建工程的一些信息，如下图所示。



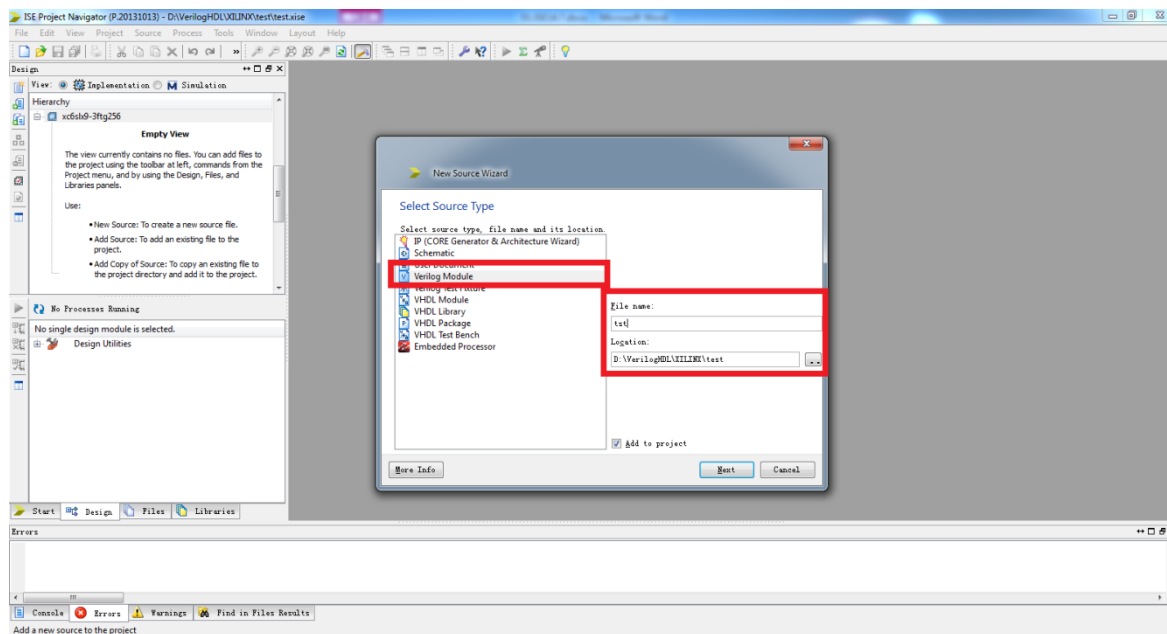
步骤五：单击 Finish 按钮，完成创建空白工程，在管理区可以看到新创建工程，如图所示。



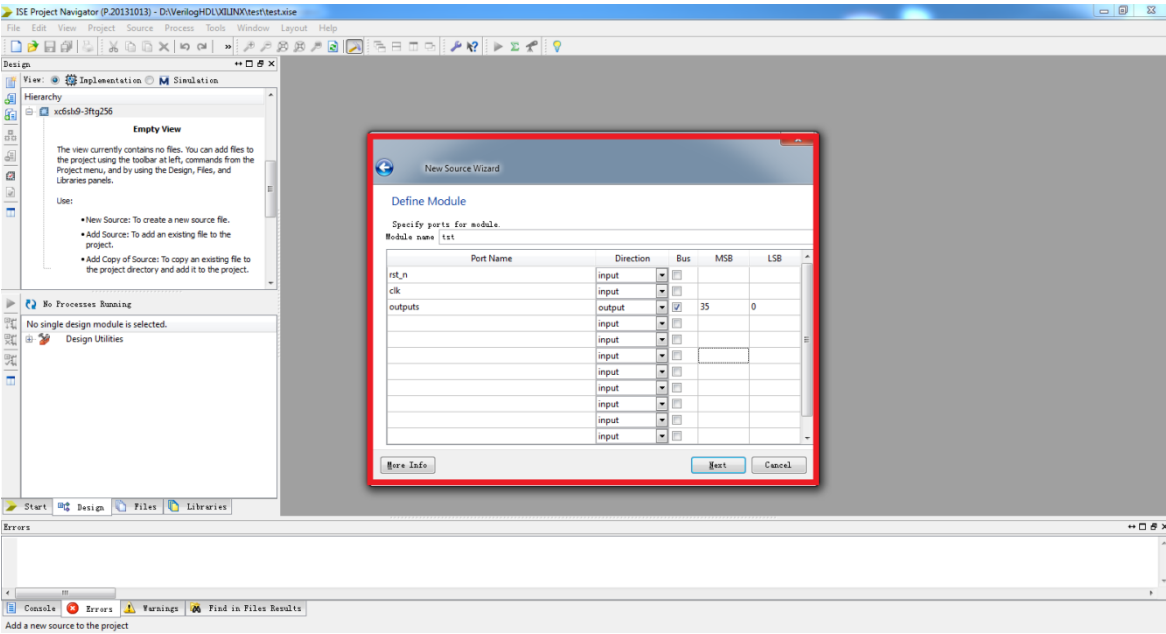
简单的工程建立完毕，现在需要给新建的工程中新建源代码。

1.1.1 新建源代码

下面开始给新工程创建并添加源文件，在管理区内右击工程 Ledtst，选择“New Source”，出现创建源文件向导的选择源文件类型对话框，如图所示，在此选择“Verilog Module”，在 File Name 文本框中输入“test”，下面有个复选框选择将文件添加到工程中。



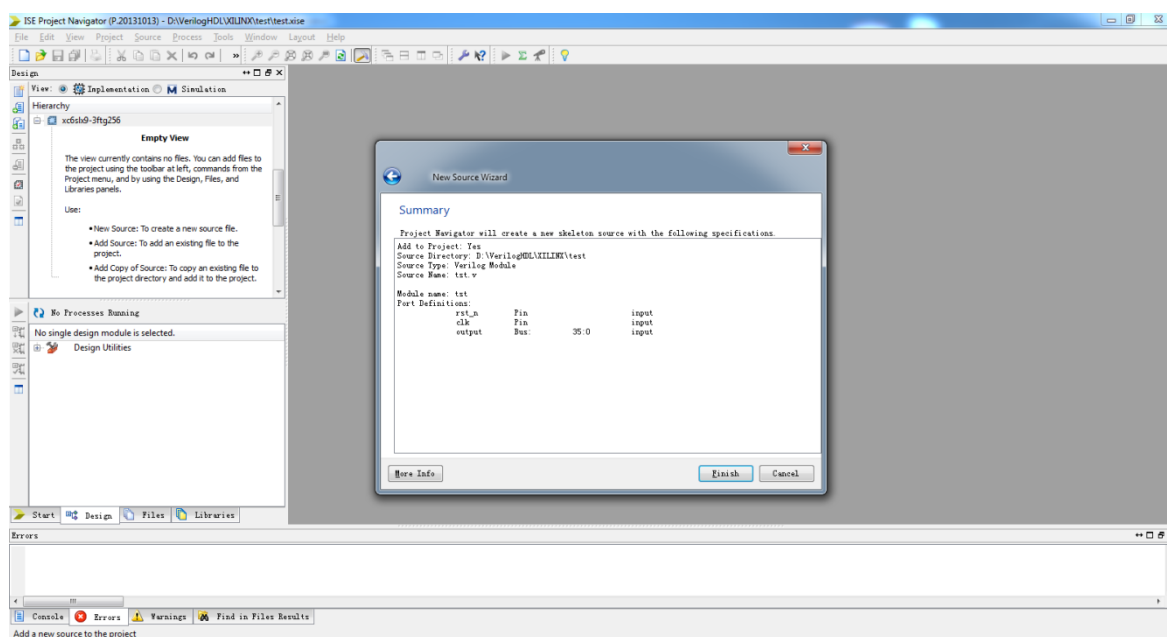
单击 Next 按钮，出现模块定义对话框，如图所示，这里实现一个这是一个 36 位彩灯依次循环点亮程序，单步模式下每次点亮一个 LED 灯，36 位 LED 灯全部点亮后从第一位循环开始。



其中，“Module Name”就是输入的“test”，下面的列表框用于对端口的定义。“Port Name”表示端口名称，“Direction”表示端口方向(可以选择 input、output 或 inout)，
“MSB”表示信号的最高位，“LSB”表示信号的最低位。单位信号的 MSB 和 LSB 不用填写。
在端口定义对话框中可以先不做任何定义，直接 Next。

定义了模块端口后，单击“Next”按钮进入下一步，单击“Finish”按钮完成创建。这样，ISE 就会自动创建一个 VHDL/Verilog HDL 模块的例子，并且在源代码编辑区打开。简单的注释、模块和端口定义已经自动生成，所剩余的工作就是在模块中实现代。

单击 Next 按钮，出现新文件总结对话框，如图所示。



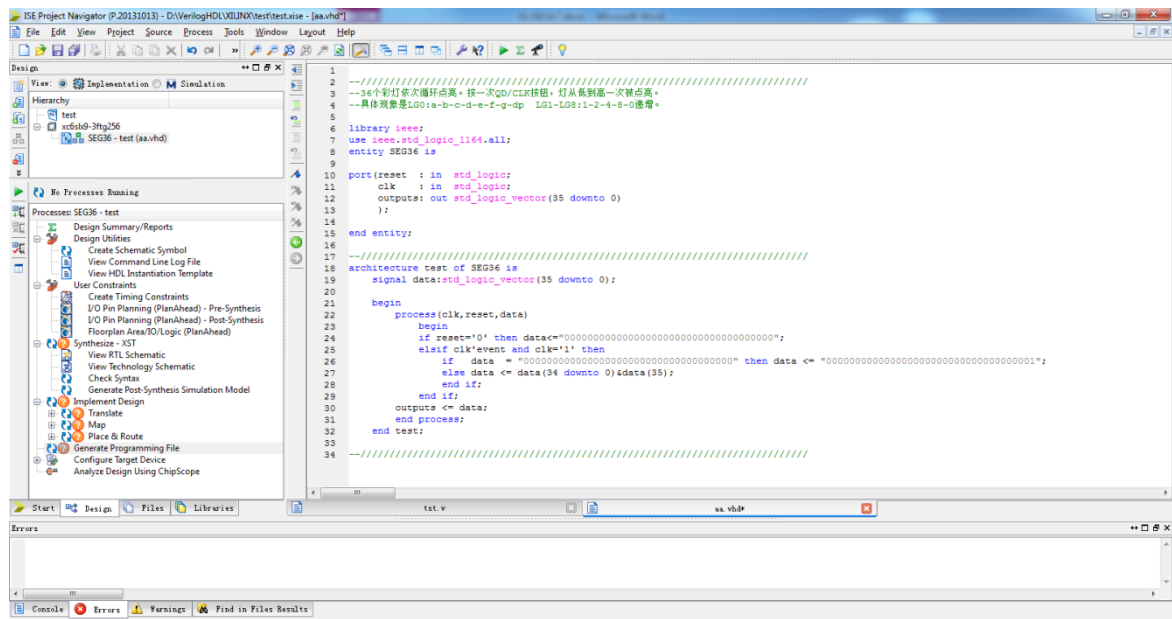
单击 Finish 按钮完成新文件创建，在管理区能看到新文件的名字，在编辑区内是 VHDL/Verilog 代码，但只包含了框架部分。自行在编辑区内修改 VHDL/Verilog 源代码，具体代码如下：

```

1
2  --////////////////////////////////////
3  --36个彩灯依次循环点亮。按一次QD/CLK按钮，灯从低到高一次被点亮。
4  --具体现象是LG0:a-b-c-d-e-f-g-dp  LG1-LG8:1-2-4-8-0递增。
5
6  library ieee;
7  use ieee.std_logic_1164.all;
8  entity SEG36 is
9
10     port(reset : in std_logic;
11          clk : in std_logic;
12          outputs: out std_logic_vector(35 downto 0)
13          );
14
15 end entity;
16
17 --////////////////////////////////////
18 architecture test of SEG36 is
19     signal data:std_logic_vector(35 downto 0);
20
21     begin
22         process(clk,reset,data)
23         begin
24             if reset='0' then data<="00000000000000000000000000000000";
25             elsif clk'event and clk='1' then
26                 if data = "00000000000000000000000000000000" then data <= "00000000000000000000000000000001";
27                 else data <= data(34 downto 0)&data(35);
28                 end if;
29             end if;
30             outputs <= data;
31         end process;
32     end test;
33
34  --////////////////////////////////////

```

编写代码后保存，自动生成为工程的 Top 文件，新文件添加完成如下图所示：



1.1.2 添加源代码

小工程，所以代码直接在上一步添加完成。

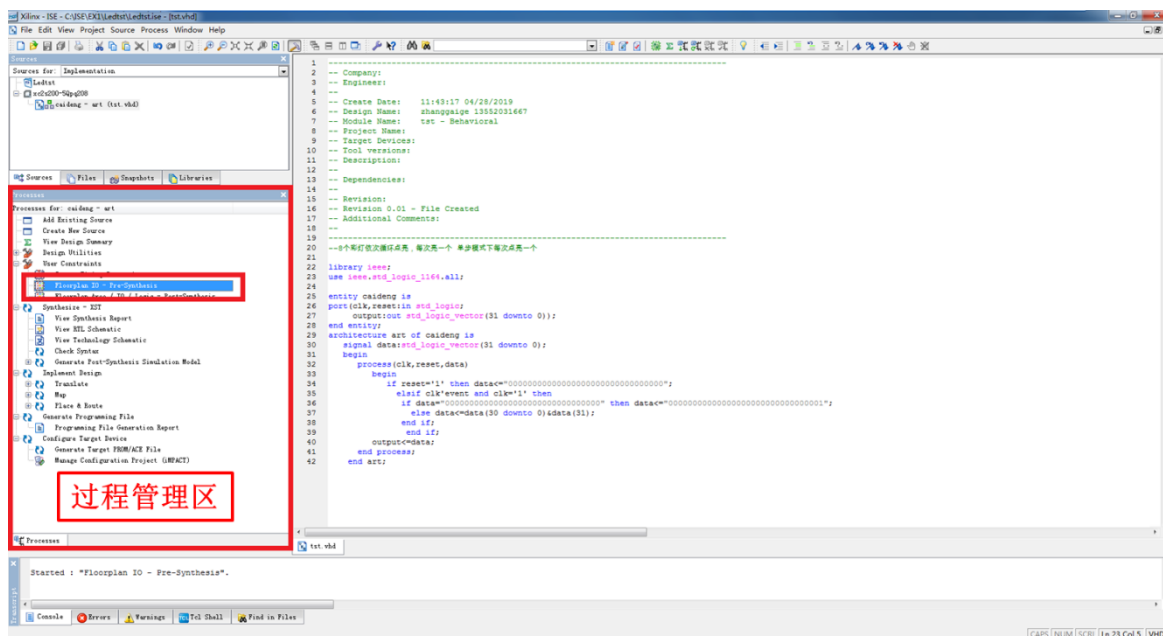
1.1.3 代码模板

ISE 中内嵌的语言模块包含了大量的开发实例和所有 FPGA 语法的介绍和举例，包含 Verilog HDL/HDL 的常用模块、FPGA 原语使用实例、约束文件的语法规则以及各类指令和符号的说明。语言模块不仅可在设计中直接使用，还是 PFPGA 开发的最好的工具手册。

1.2 添加约束

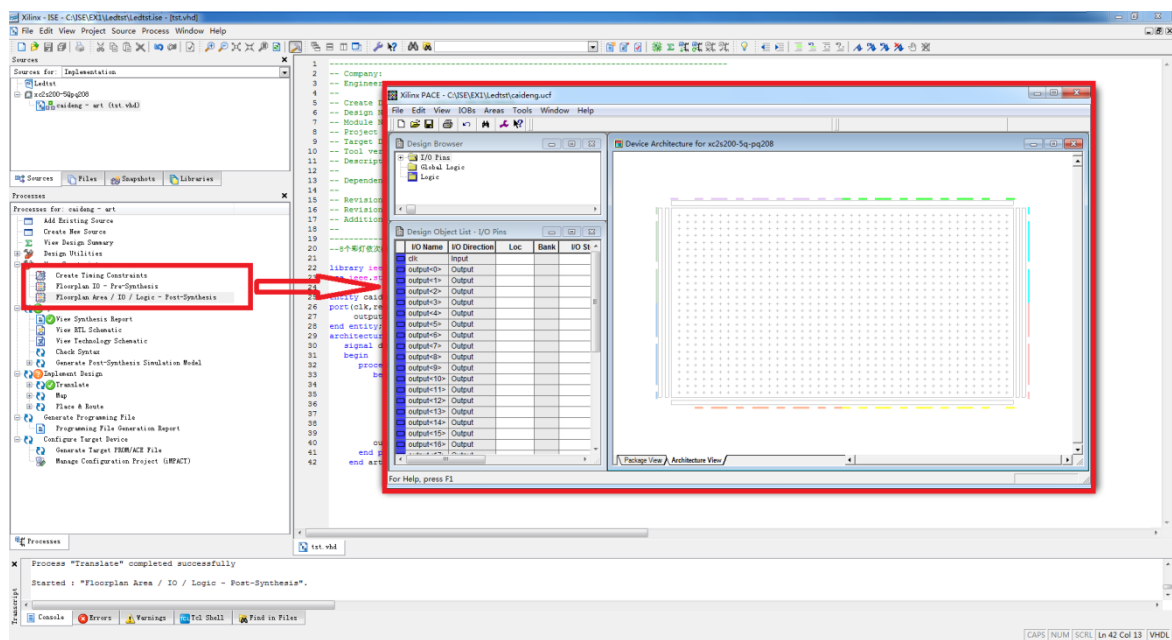
约束是 FPGA 设计所不可缺少的。例如，管脚约束将模块的端口和 FPGA 的管脚对应起来。在 ISE 中有很多种约束，它们可以指定设计各个方面的设计要求。通过附加约束可以控制逻辑的综合、映射、布局和布线，以减少逻辑和布线延时，从而提高工作频率，还可以指定 I/O 引脚所支持的接口标准和其电气特性等，本节只介绍如果对设计进行管脚约束。这里我们需要对 Ledtst.vhd 程序中的输入输出端口分配到 FPGA 的真实管脚上，这里需要准备一个 FPGA 的引脚绑定文件.ucf 并添加到工程中。

在管理区的“View”栏中选择“Implementation”项，选择要约束的源文件，这里是“SEG36.vhd”，在下面的操作窗口中展开“User Constraints”，会出现用户约束设定的选项，如图所示。



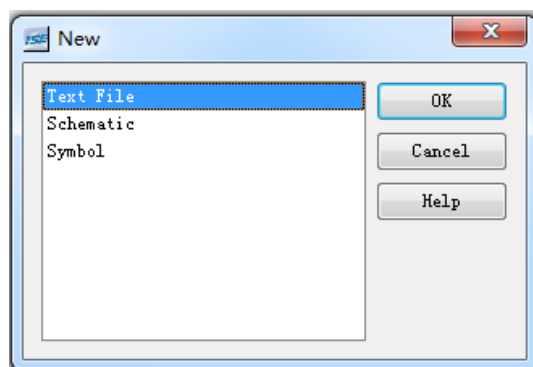
双击“I/O Pin Planing-Post-Synthesis”，会出现添加用户约束文件的对话框，单击 YES 按钮，确认添加用户约束文件，打开管脚约束窗口。

在其中可以指定设计的管脚，在左侧的窗口中选择 I/O Port 标签栏，会出现设计中所有的 I/O 管脚，展开后在 Site 列中就可以指定对应的管脚号，直接单击输入即可。管脚指定完成后保存退出窗口，在管理区的 tst.vhd 文件下会出现约束文件 tst.ucf。



用户也可以直接编辑该文件进行约束设定：

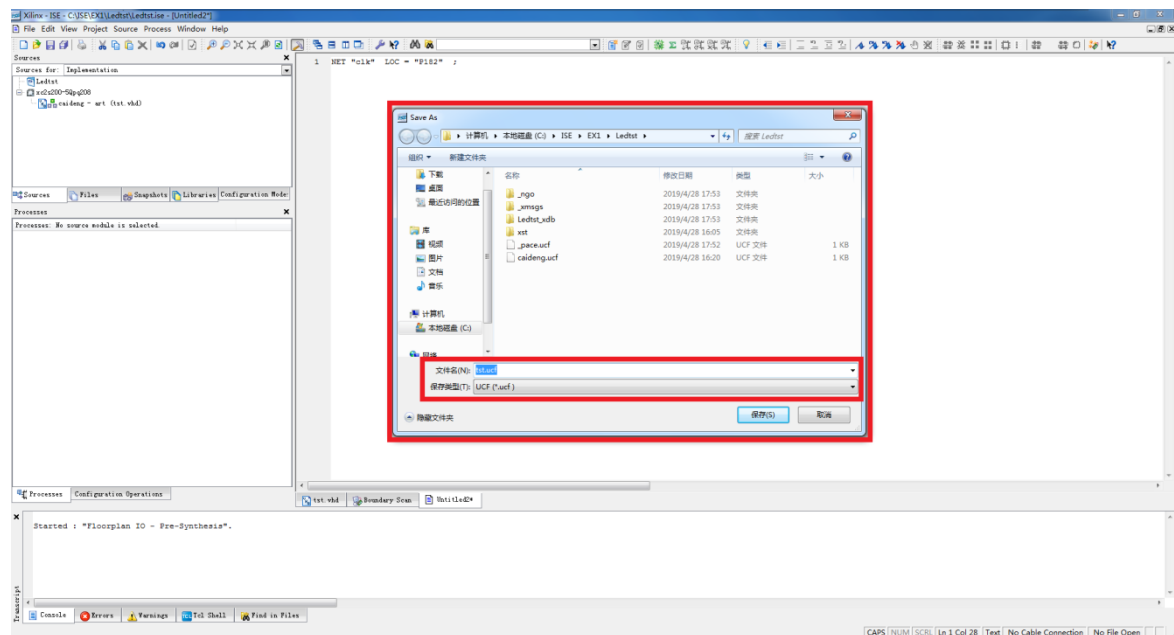
步骤一 新建一个空白文件 File->New,选择 Text File。



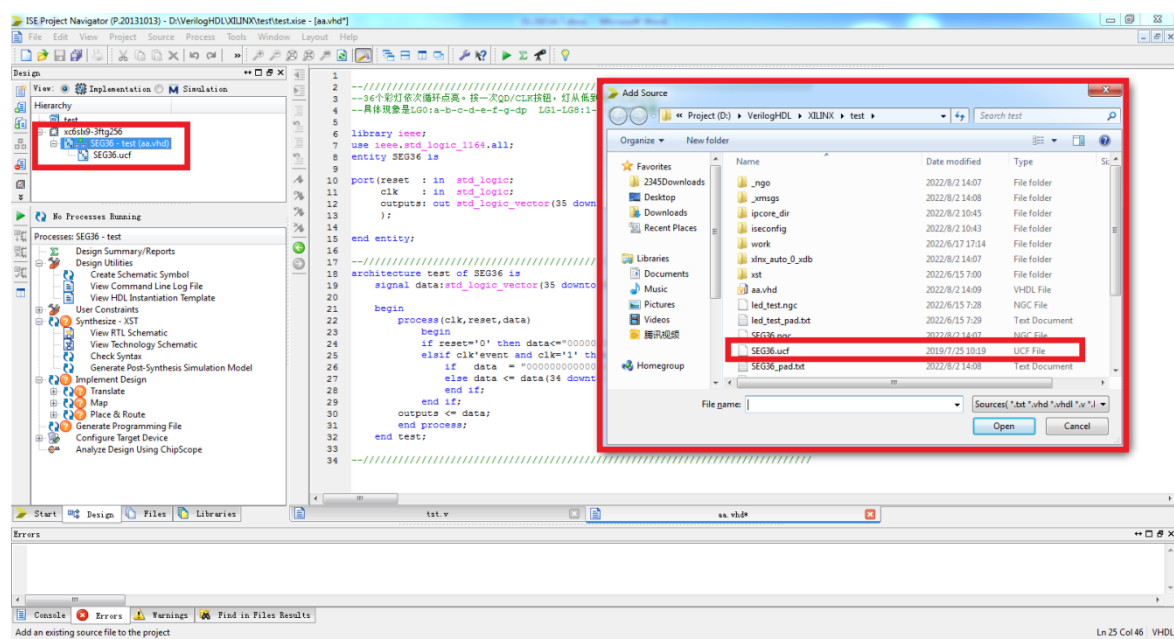
步骤二 在这里 Text 文件中添加以下的引脚定义

基本的 UCF 编写的语法，普通 I/O 口只需要约束引脚号和电压；NET “端口名称” LOC = 引脚编号 | IOSTANDARD = “电压”；例如：NET “clk” LOC = “J16”；需要注意的是，ucf 文件是大小敏感的，端口名称必须和源代码中的名字一致，且端口名字不能和关键字一样。但是关键字 NET 是不区分大小写的。

步骤三 完成后另存为 SEG36. ucf 文件



步骤四 把 SEG36. ucf 文件添加到工程中，点击 Project→Add Source, 添加后如下图所示：



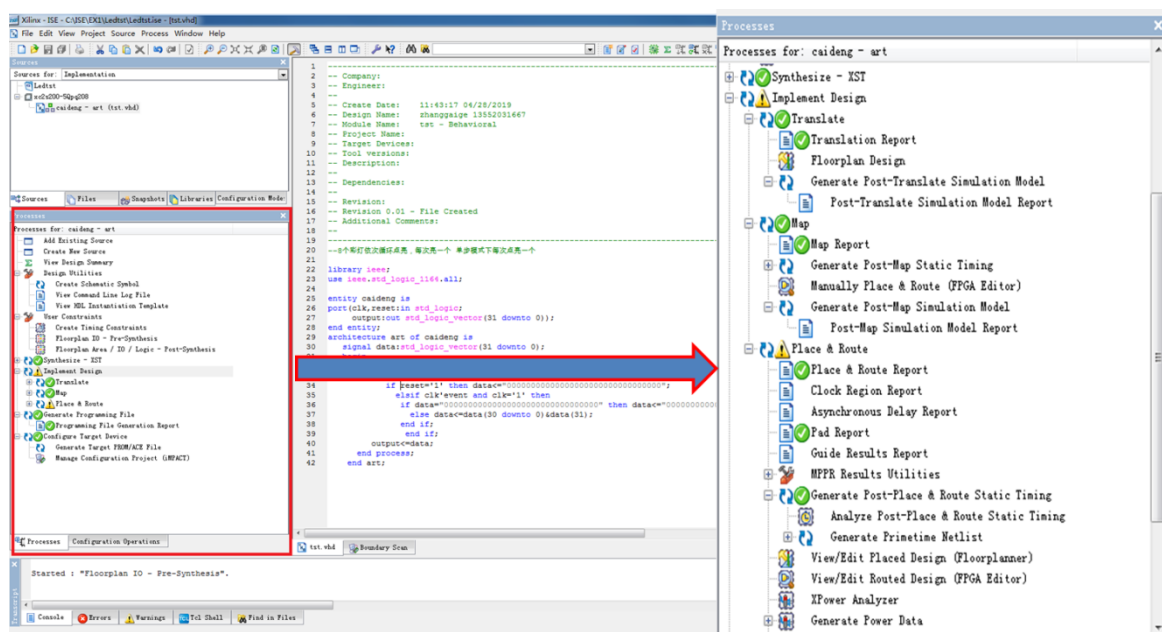
1.3 综合与实现

保存项目并且开始编译，综合与实现就是将逻辑网表翻译成底层模块与硬件原语，将设计映射到器件结构上，进行布局布线，分成翻译(Translate)、映射(Map)、布局布线(Place&Route)三步：

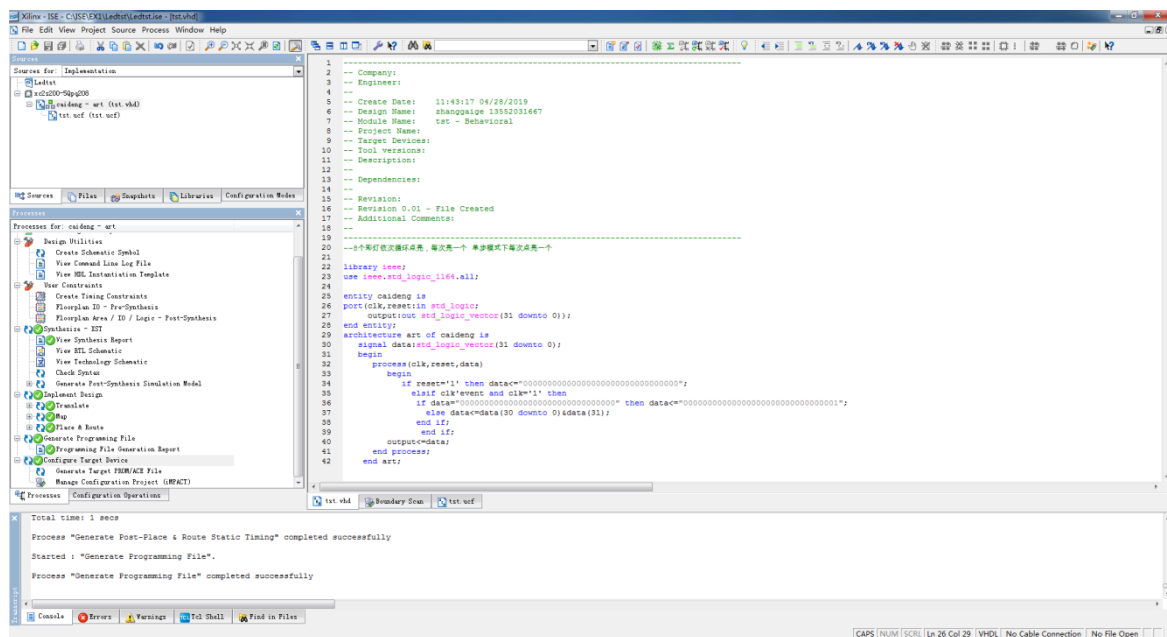
(1) 翻译：把多个设计文件合并成一个单独的网表设计。

(2) 映射：把网表中的门级逻辑映射到物理器件资源上。

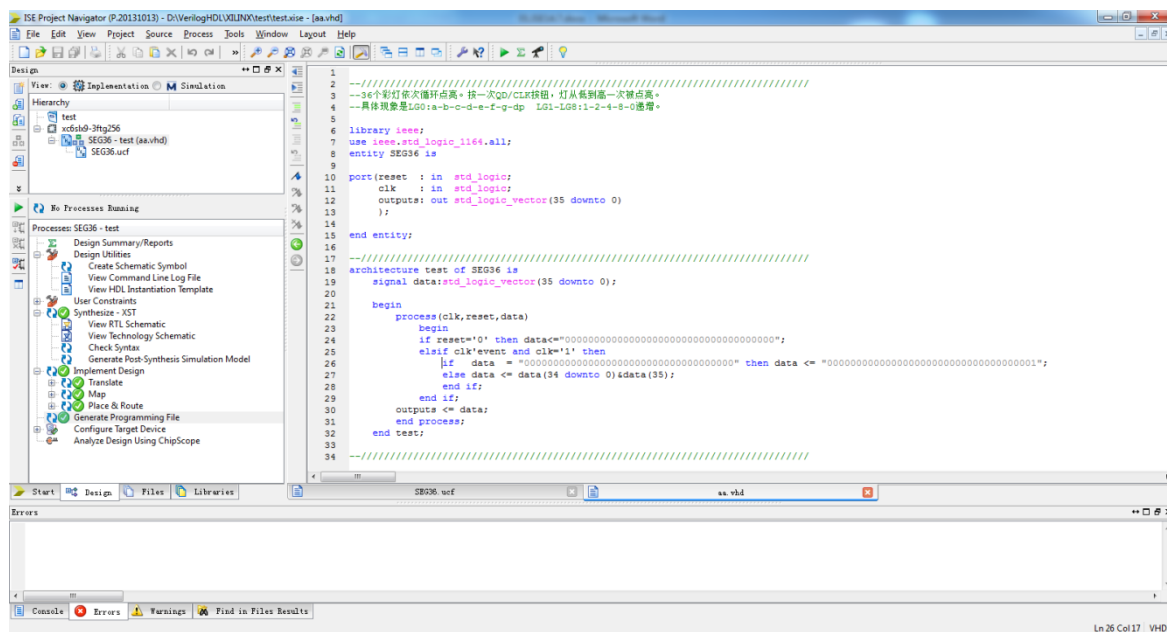
(3) 布局布线：把 Map 中的物理器件资源在器件上布局，并用布线资源连接起来，把时序数据写入到时序报告中。



在管理区中选择 SEG36. vhd, 在操作栏中选择 implement Design, 双击开始实现, 实现成功后, 设计就会有延时信息, 后续可以进行时序仿真。

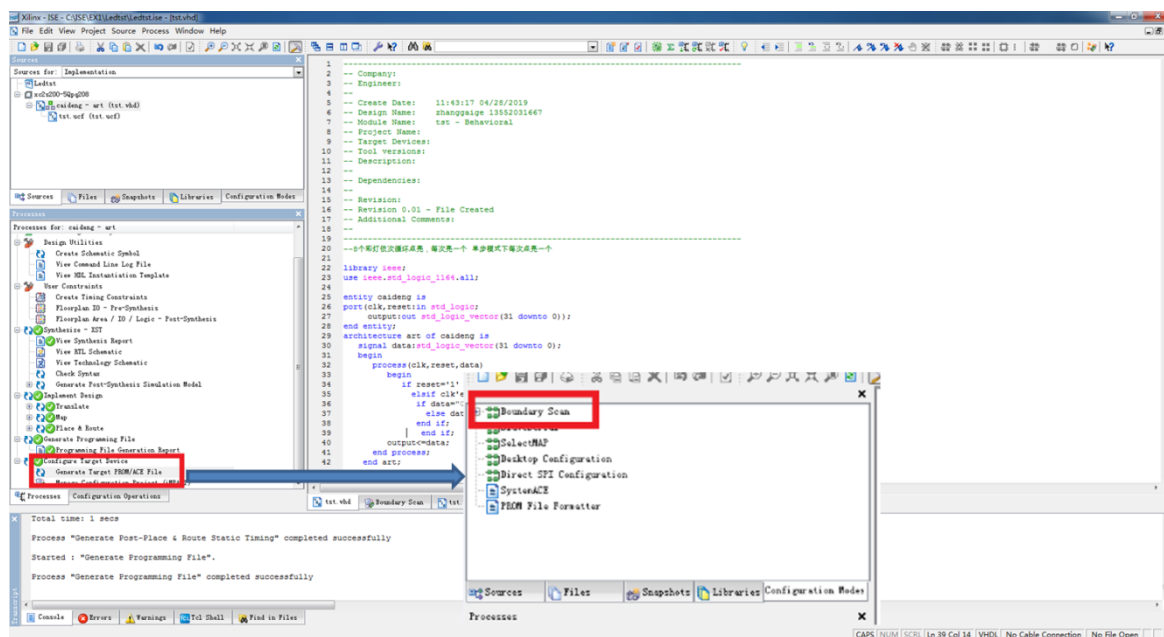


程序编写和管脚分配完成之后, 需要综合, 在软件左侧 Process 栏, 选择 Process, 双击 Synthesize Design 对设计进行编译工程, 编译完成完成后 Synthesize Design 显示绿色对勾(如果显示红色叹号, 说明代码有问题, 根据提示修改代码)。编译成功后在 Console 窗口出现成功的信息。



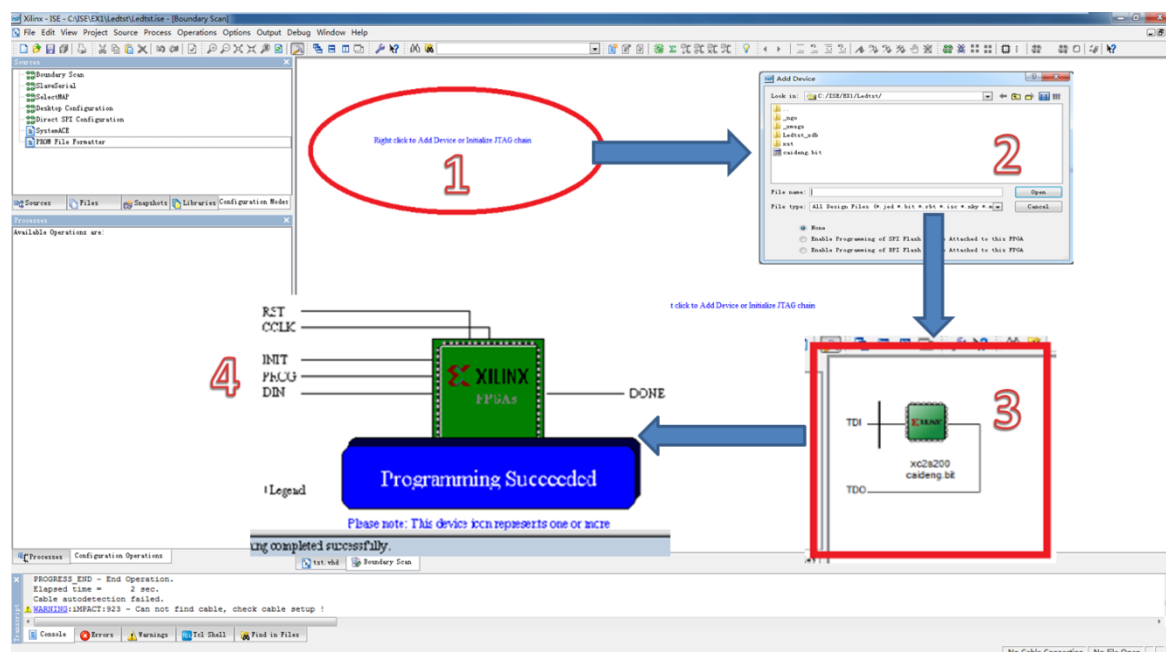
1.4 程序下载

经过前面的编译和综合，我们可以把 Bit 文件下载到 FPGA 芯片中，看一下程序实际运行的效果，随后开始实际测试了，首先在配置之前需要生成配置文件，并连接到下载电路。双击操作栏中的“Generate Programming File”，生成可配置文件，将 USB 下载线分别和计算机接口及实验仪的 JTAG 口相连接，首次连接计算机会自动安装驱动程序，然后打开设备供电。双击操作栏的 Configure Target Device，会弹出询问是否打开 IMPACT 的对话框，单击 OK 按钮，将打开 IMPACT 窗口。



双击左边窗口中的 Boundary Scan, 在右边出现的空白窗口中右击选择 Initialize Chain。

iMPACT 会识别出电路板上的 FPGA 芯片, 显示出一个 FPGA 芯片图标, 并弹出是否制定配置文件对话框, 单击 Yes 按钮, 弹出制定配置文件对话框, 在其中选择刚才生成的配置文件 tst.bit。



配置成功后, 单步模式下每次点亮一个 LED 灯, 32 位 LED 灯全部点亮后从第一位循环开始。截止到这一步, 我们已经完成了熟悉 FPGA 的基本流程。